Module (3)

2/4/26

DSA - Java

Day (1)

thinking in algorithms

Algorithm to define addition of two number

start

↓

int a, b;

↓

int a = 10;

↓

int b = 20;

↓

int c = a + b;

↓

end

complexity	Name
O(1)	constant
O(logn)	logarithmtic
O(n)	linear
O(nlogn)	linearimthic
O(n^2)	Quadritic
O(2n)	Eaponential
O(n!)	Factorial

Module (3)

26/9/26

DSA - Java

Day (1)

thinking in algorithms

Algorithm to define addition of two number

start

↓

int a, b

↓

int a = 10;

int b = 20;

↓

int c = a + b;

↓

end

complexity	Name
O(1)	constant
O(logn)	logarithmtic
O(n)	linear
O(nlogn)	linearimthic
O(n^2)	Quadritic
O(2^n)	Eaponential
O(n!)	Factorial

Module (3)

26/9/26

DSA - Java

Day (1)

thinking in algorithms

Algorithm to define addition of two number

start

int a, b

int a = 10;

int b = 20;

int c = a + b;

end

complexity	Name
O(1)	constant
O(logn)	logarithmtic
O(n)	linear
O(nlogn)	linearimthic
O(n^2)	Quadritic
O(2n)	Eaponential
O(n!)	Factorial

single → O(n)
nested → O(n²)
triangle → O(n²)
halving → O(log n)
nlogN → O(N log n)
sequential → O(n)
sqrtloop → O(n)
sqrtloop → O(√n)
constant → O(1)

Day② - ADS using Java

27/9/26

In Java we can't get exact address bcz Java does not support to the pointers

	1000	1004	1008	1012	1016	1020
arr marks =	12	45	7	33	90	18
	0	1	2	3	4	5

address of marks[3] = 1000 + 3 x 4
= 1012

one multiplication and one addition whetaver size of array O(1)

rray index → O(1)

Dry run codes Day ②

M T W T F S S
Page No.:
Date:
YOUVA

```
Static void insertAt (int[] a, int size,
                      int index, int value) {
For (int i = size; i > index; i--) {
    a[i] = a[i-1];
}
a[index] = value;
}
```

⇒ insertAt (a, 4 elements, insert 15 at index);

a = [10, 20, 30, 40, 0]
5 position → because we need one empty space for new element

we want value 15 at index 1
Answer should be [10, 15, 20, 30, 40]

For (int i = size; i > index; i--)

size = 4, index = 1
i = 4 & 4 > 1 ✓ → 4 → 3 → 2

When i = 1 i > 1 NO

So i goes 4, 3, 2

First iteration

a[i] = a[i-1]

i = 4

a[4] = a[3]

index :- 0 1 2 3 4

10 20 30 40 0

copy 40 a[4] = a[3]

10 20 30 40 40

second :- i = 3

a[3] = a[2]

30 -> index 3

10 20 30 30 40

ind :- i = 2

a[2] = a[1]

10 20 20 30 40

Now a[index] = value

a[1] = 15

done ✓

ay. copyOF() -> /* create big array
nd copy the old elements */

void grow() → make array bigger

M T W T F S S

Page No.:

Date:

YOUVA

code ==

dynamic array

```
public void add(T item){
    if(size == data.length)grow();
data[size++] = item;
}

public void grow(){
   int oldcap = data.length;
int newcap = math.max(oldcap + (oldcap >>1), oldcap +1)
data = Arrays.copyOF(data, newcap);
}
```

data ≈ {10, 20, 30, 40}

size = 4

index = 1

data[1] = data[2]

10, 20, 30, 40 → 10 30 30 40

0 1 2 3

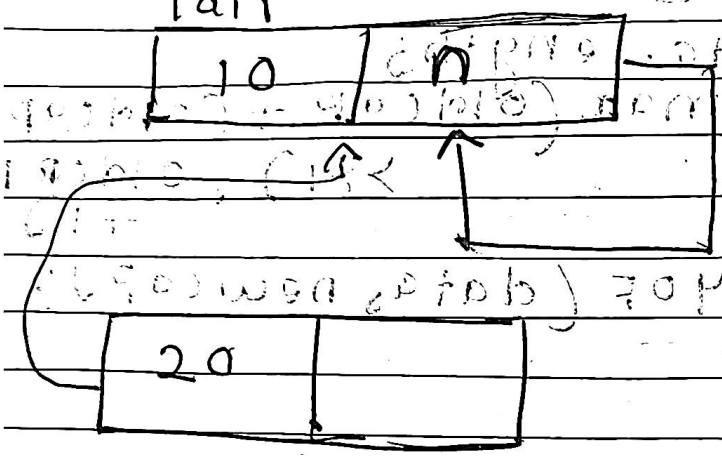
loop (2) i = 2

data[2] = data[3]

[10, 30, 40, 40]

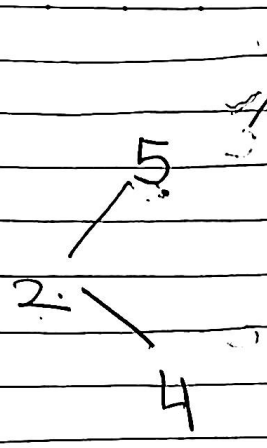
size--;

circular link list



Tree

10, 5, 2, 4, 20, 30



pre order → 10, 5, 2, 4, 20, 30

Inorder (left, root, right)

5, 2, 4, 10, 20, 30

postorder → 4, 2, 5, 30, 20, 10

preorder → 10, 5, 2, 4, 20, 30

preorder (Root, left, right)

Array → O(1)

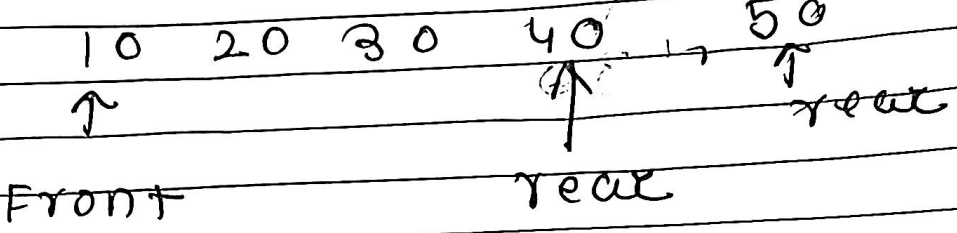
rear →

Front →

queue

enque deque

insert O delete



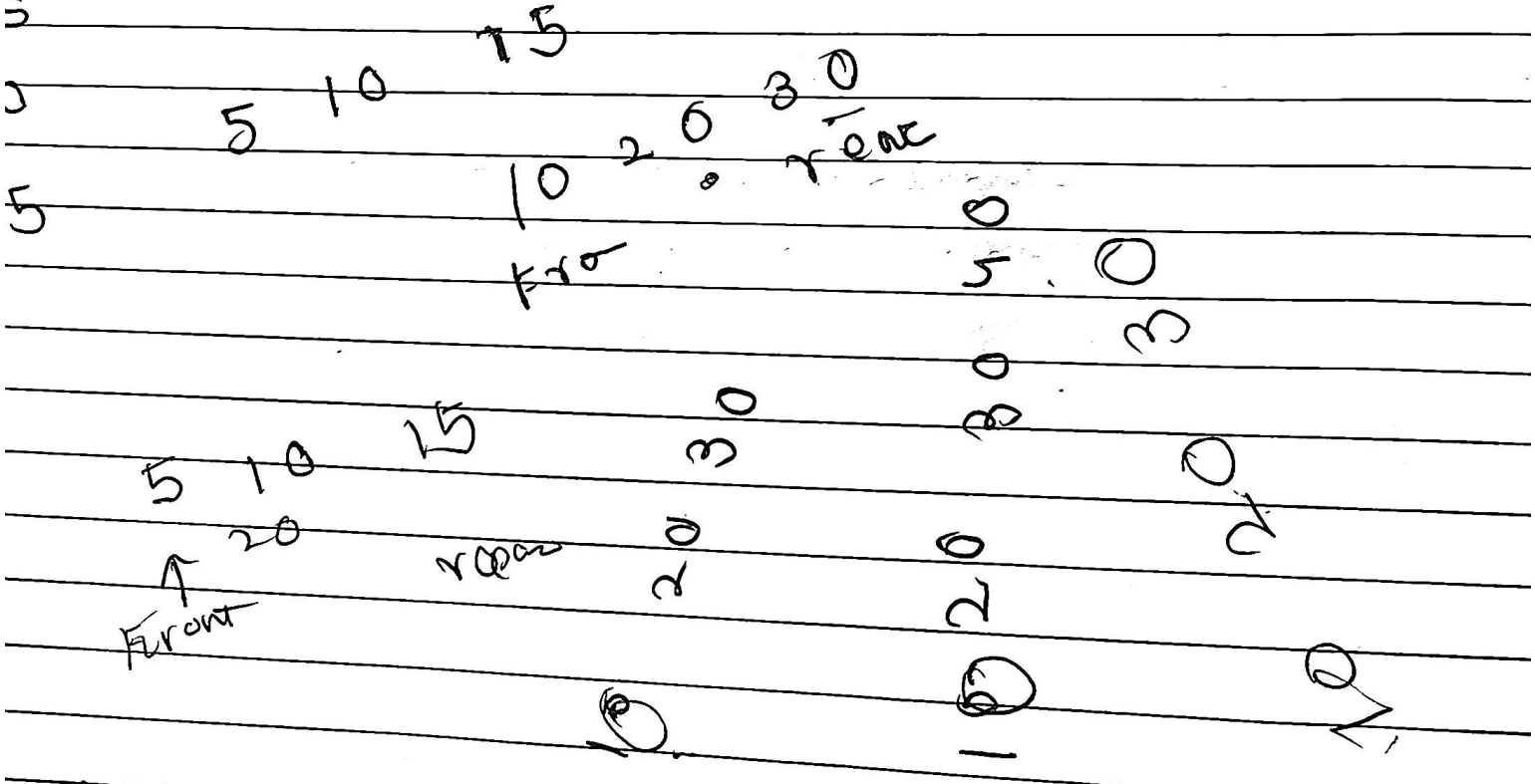
enque → add at rear

Deque → reme from Front

enque → rear

deque → Front

peek → front



rear formula

rear = (Front + size - 1) % capacity;

next free position → (rear + 1) % capacity

(5 + 5 - 1) % 8

10 - 1

9 % 8

8 / 9

8

1

(4 + 1) % 5

5 % 5

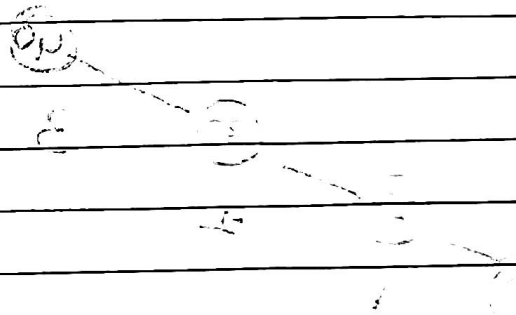
1

(4 + 3 - 1) % 6

7 - 1

6

Circular queue



Day 6

AVL tree

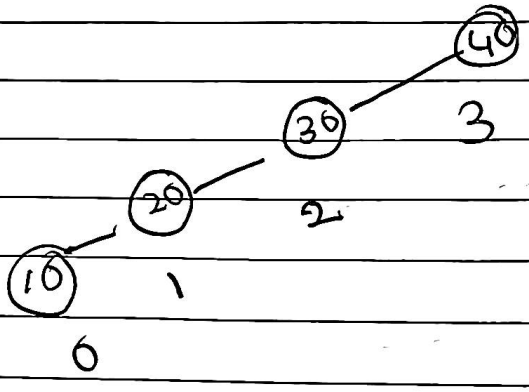
AVL tree is self balancing binary search tree

BST rule -> left child is smaller than the parent right child is greater

Balance rule -> height diff between left and right subtree of every node cannot be more than 1

Balance Factor = H(left) - H(Right)

-1, 0, +1



Four type of AVL rotation when node become unbalanced.

LL

RR

R

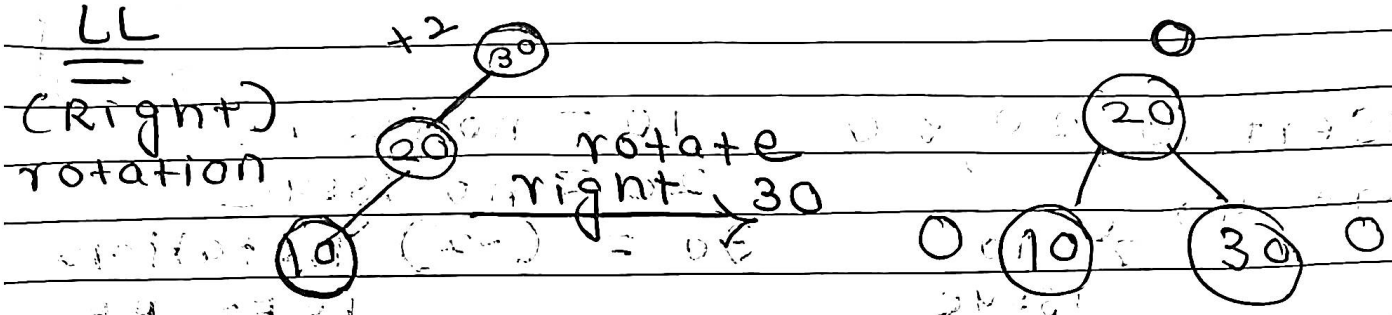
L

LL Rotation (left Rotation)
RR Rotation (Right)

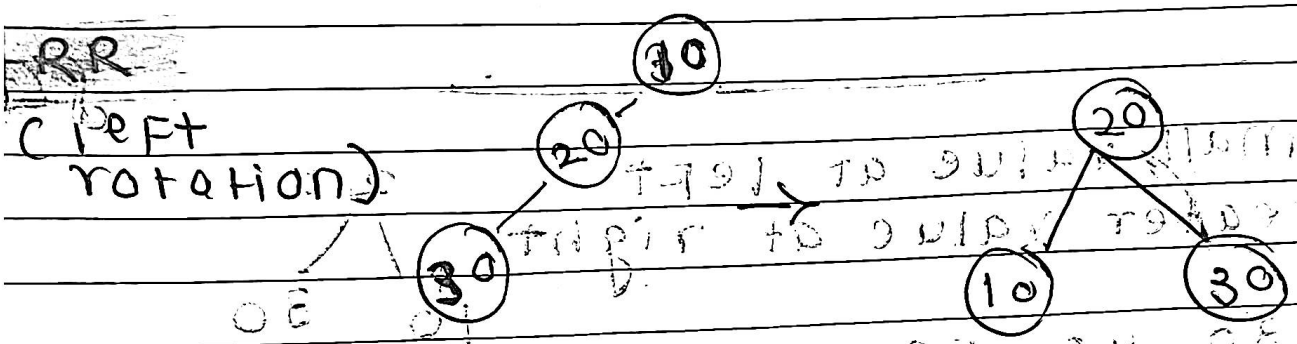
LR more than 1 rotation 1st left and then right

RL more than one rotation 1st right and then left

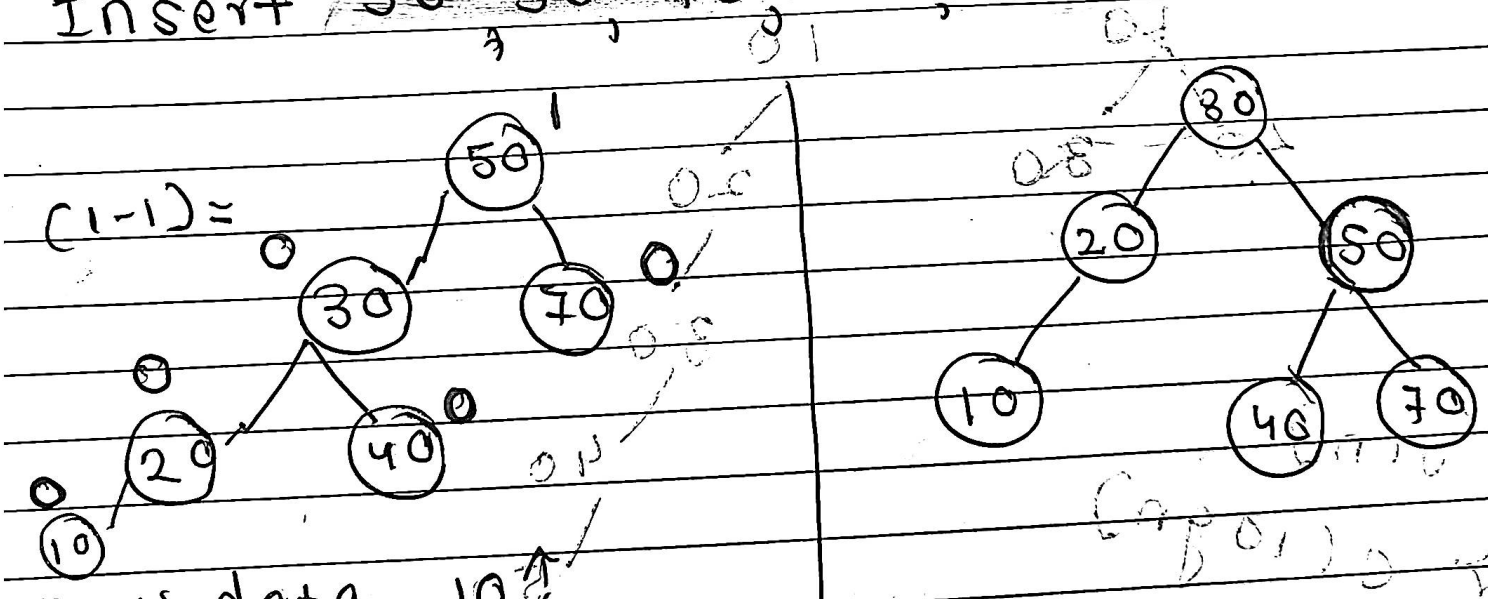
LL
(Right rotation)



RR
(left rotation)



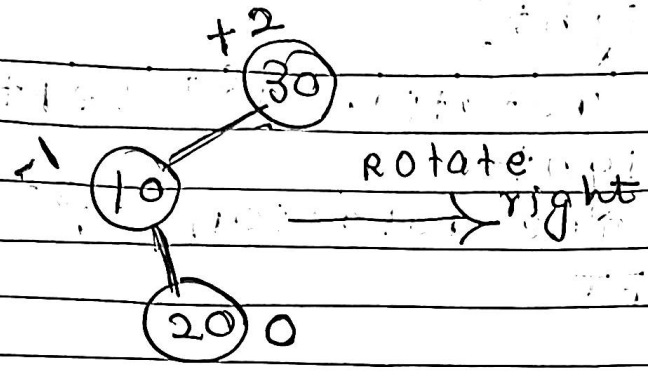
Insert 50, 30, 70, 20, 40



new data 10

LR

insert 30 10 20



Q

insert 10, 20, 30

insert 40 → no issue

50.

10 = no issue

20 = no issue

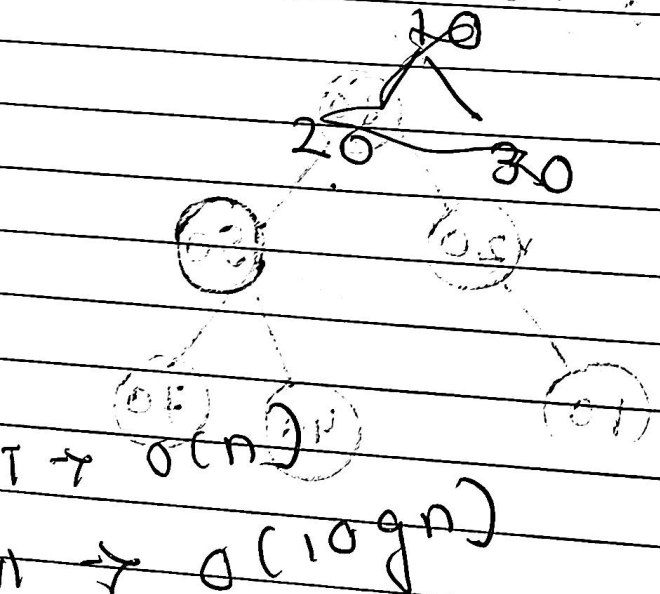
30 = (-2) Rotation left RR case

BST

Small value at left

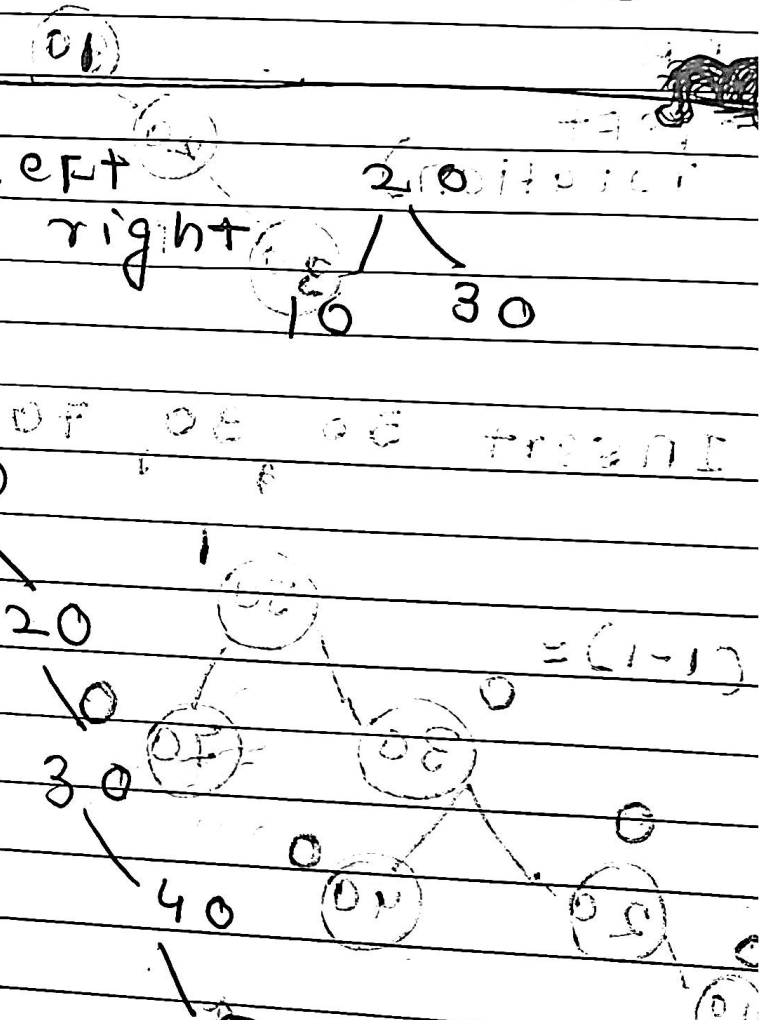
Greater value at right

10, 20, 30, 40, 50



T → O(n)

h → O(log n)



+ → left heavy
- → right heavy

+2 → too much left

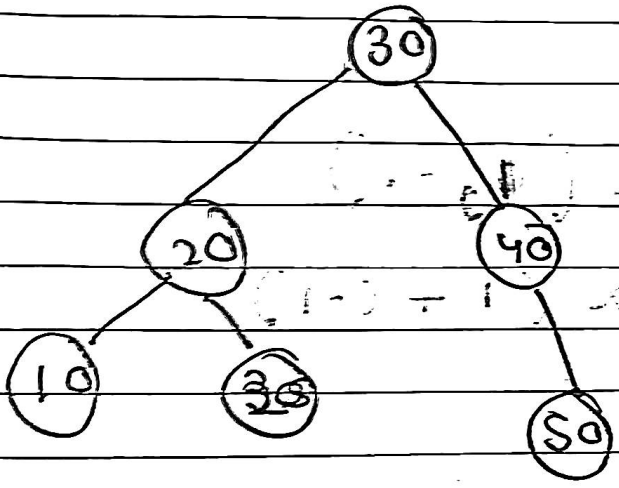
-2 → too much right

AVL → Adelson velsky and landis

Balance Factor = height(left) − height(right)

Empty subtree has height -1

leaf has height 0



10 has no children

left = empty = -1

right = empty = -1

height = 1 + max(left height, right height)

1 + max(-1, -1)

1 + (-1)

② 20 → 10 height = 0

10 left = 0 height -1

height (20)

1 + max (0, 1)

1 + 0

= 1

20 height ①

10 height 0

30

20 ⇒ 10 → height 0

20 → height 1

10 30 →

height (30)

⇒ 1 + max (, -1)

1 + max (1 + (-1)

⇒ 1

there are 3 node

Node 30

Node 20

Node 40

30

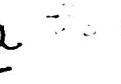
20 40

class Node {

int key, height ;

Node left, right ;

ex



① $h(10) = 0$ 10 is a leaf

② $h(30) = 0$ 30 is a leaf

③ balance (20) = left height − right height

$= 0 - 0 = 0$

④ height (20) = $1 + \text{math.max}(h(n.\text{left}), h(n.\text{right}))$

$= 1 + \max(h(10), h(30))$

$= 1 + \max(0, 0)$

$= 1 + 0$

$= 1$

max node = $2^{h+1} - 1$

max node in a complete binary tree of height 5 (edge)

$\Rightarrow 2^{5+1}$

$\Rightarrow 2 = 2 \times 2 \times 2 \times 2 \times 2 \times 2 = 63$

Algorithm	Best	Avg	Worst
Selection	$o(n)^2$	$o(n^2)$	$o(n^2)$
Insertion	$o(n)$	$o(n^2)$	$o(n^2)$
Bubble	$o(n)$	$o(n^2)$	$o(n^2)$
merge	$o(n\log n)$	$o(n\log n)$	$o(n\log n)$
quick	$o(n\log n)$	$o(n\log n)$	$o(n^2)$
heap	$o(n\log n)$	$o(n\log n)$	$o(n\log n)$

Page No.: YOUVA
Date:

Day 47 / Sorting

☆ Selection sort

Bubble

[4, 5, 2, 3] compare adjacent element

```
Code
class Sorting {
public static void main (String [] args) {
int arr[] = {7, 8, 3, 1, 2};
for (int i=0; i< arr.length-1; i++) {
```

```
For (int J=0; J< arr.length-1-I; J++) {
    if (arr[J] > arr[J+1]) {

        // swap
        int temp = arr[J];
        arr[J] = arr[J+1];
        arr[J+1] = temp;
    }
}
```

Selection sort

[4, 8, 3, 1, 2]

```
for (int i=0; i< arr.length; i++) {
    int smallest = i;

for (int J= i+1; J< arr.length; J++) {

if (arr[smallest] > arr[J]) {

    smallest = J;
}
}
    int temp = arr[smallest];
    arr[smallest] = arr[i];
    arr[i] = temp;
}
```

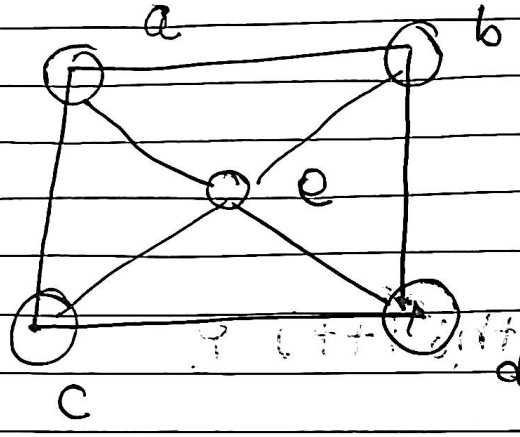
~~Insertion sort~~

Page No.: Date: YOUVA

Graph

A graph is (V, E) is set of vertices and edges where we write $|V| = V$ and $|E| = E$

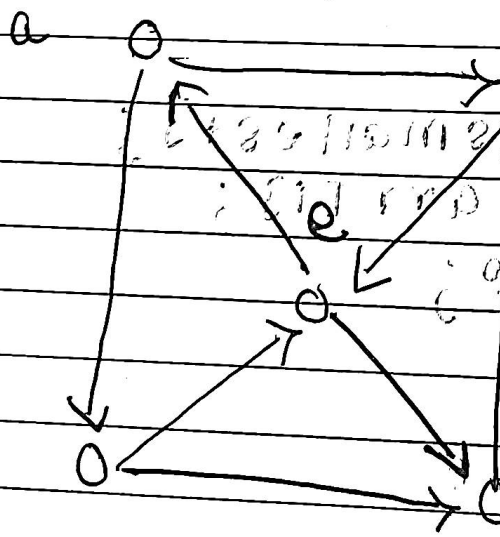
undirected (edge work both way)



	a	b	c	d	e
a	0	1	1	0	1
b	1	0	0	1	1
c	1	0	0	1	1
d	0	1	1	0	1
e	1	1	1	1	0

degree of a, b, c, d = 3

directed graph



a → in (1)
out (2)

e → in (2)
out (1)

d → in (2)
out (1)

e → in (0)
out (2)

b → in (1)

	a	b	c	d	e
a	0	1	1	0	0
b	0	0	0	1	1
c	0	0	0	0	1
d	0	0	1	0	0
e	0	0	1	0	0

M T W T F S S

Page No.:

Date:

YOUVA

✱ Selection sort

```
For (int i=0 ; i< a.length -1 ; i++) {
    int min = i;
    For (int J= i+1 ; J< a.length ; J++) {
        iF( a[J] < a[min]) {
            min = J;
        }
    }
    int temp = a[i];
    a[i] = a[min];
    a[min] = temp;
}
```

0 1 2 3 4

int[a] = 6 3 8 1 5

min = i = 0 = 6

temp = a[i] = a[0] = 6

a[i] = a[min]

a[3] = a[0] = 6

3 < 6

yes

min = 3

0 < 3 NO = min = 3

1 < 3 yes

min 1

Insertion sort

{5, 3, 4}

```
import java.util.Arrays;
public class main {
static void insertionSort(int[] a) {
for(int i=1; i<a.length; i++) {
    int key = a[i];
int j = i-1;
while (j>=0 && a[j] > key) {
    a[j+1] = a[j];
    j--;
}
a[j+1] = key;
}
```

element 5 is already sorted

key = a[1] = a[1] = 3 key = 3

[5] [3] [4]
 ↑
 key

j = i-1 = 1-1 = 0